

Parallel and Distributed Computing

Lab 07 — Chapter 9: Parallel Sorting (Problems 9.1–9.10)

Practical Report (Code, Execution, Results & Analysis)

Name: Huzaifa Arshad

Registration No: 2023-CS-86

Date: 25 May 2026

Build & Run Environment

All MPI programs were compiled with mpicc (OpenMPI) and the OpenMP programs with gcc -fopenmp. Each problem lives in its own directory in the accompanying code archive (prob9_1 ... prob9_10) with a Makefile and README.

Common requirements

- MPI: mpicc / mpirun (OpenMPI or MPICH) — used by 9.1, 9.4–9.10.
- GCC with OpenMP (-fopenmp) — used by 9.2 and 9.3.
- Python 3 + matplotlib + numpy — only for the 9.9 efficiency plot.

Running more processes than cores

Several problems request 16 or 32 processes. On a single machine add --oversubscribe to mpirun. If MPI runs as root in your environment, also add --allow-run-as-root.

Timing figures in this report were measured on a single multi-core node with oversubscription and are illustrative; re-run on the target machine to populate final numbers.

Problem 9.1 — Compare-Split: Merge-Split vs Pairwise Exchange

Overview

Two MPI programs, each run with 2 processes, perform one compare-split. Method A exchanges the whole sorted block once, merges, and keeps its half. Method B sends one element at a time (per the hint): P_i sends x_i , P_j returns $\max(x_i, y_i)$, keeps the smaller; an early-stop fires once $x_i \geq y_i$.

Both verify the compare-split property and report execution time, message count and bytes transferred (summed over both processes).

Compilation & Execution

Compile:

```
make
```

Run:

```
mpirun --oversubscribe -np 2 ./methodA 1000
```

```
mpirun --oversubscribe -np 2 ./methodB 1000
```

Results

k	A time (s)	A msgs	A bytes	B time (s)	B msgs	B bytes
10	0.000016	2	80	0.000023	18	72
100	0.000014	2	800	0.000157	189	756
1000	0.000035	2	8000	0.001079	1527	6108
10000	0.000315	2	80000	0.010473	14784	59136

Table 9.1: Time, message count and bytes for both compare-split methods (2 processes).

Analysis

Method A sends a constant 2 messages regardless of k , while Method B sends $O(k)$ tiny messages. Even though Method B transfers fewer bytes (its early-stop halts before all k elements are exchanged), it is up to $\sim 15\times$ slower at $k=10000$ because per-message startup latency (t_s) dominates — each element pays a full message startup.

MIMD vs SIMD: Method B is MIMD-friendly — the two processes run different code (sender vs comparator) and use a data-dependent early-stop, i.e. independent control flow. It maps poorly onto SIMD lockstep execution. Method A's bulk transfer plus identical local merge is comparatively more SIMD-amenable.

Source Code

MethodA_mergesplit.c

```
/* Problem 9.1 - Method A: Standard merge-split compare-split (MPI, 2 processes)
 *
 * Each of the two processes holds a sorted block of k random integers.
 * P0 keeps the smaller half, P1 keeps the larger half (a compare-split).
 *
 * Method A works by exchanging the WHOLE block once, merging the two sorted
 * blocks locally, and then each process keeps its half of the merged array.
 *
 * Messages: 2 (one each direction). Bytes: 2 * k * sizeof(int).
 *
 * Build: mpicc -O2 -o methodA methodA_mergesplit.c
```

```

* Run:      mpirun -np 2 ./methodA <k>
*/
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2) {
        if (rank == 0) fprintf(stderr, "This program requires exactly 2 processes.\n");
        MPI_Finalize();
        return 1;
    }

    int k = (argc > 1) ? atoi(argv[1]) : 1000;
    if (k <= 0) k = 1000;

    int *mine = malloc((size_t)k * sizeof(int));
    int *other = malloc((size_t)k * sizeof(int));
    int *merged = malloc((size_t)2 * k * sizeof(int));

    /* Each process seeds differently so the data differs. */
    srand(12345 + rank * 777);
    for (int i = 0; i < k; i++) mine[i] = rand() % 100000;
    qsort(mine, k, sizeof(int), cmp_int); /* blocks start sorted */

    int partner = 1 - rank; /* the other process */
    long messages = 0, bytes = 0;

    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    /* Exchange the entire block (one send + one recv). */
    MPI_Sendrecv(mine, k, MPI_INT, partner, 0,
                 other, k, MPI_INT, partner, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    messages += 1; /* this process sent one message */
    bytes += (long)k * sizeof(int);

    /* Merge two sorted arrays of length k into 'merged' (length 2k). */
    int i = 0, j = 0, m = 0;
    while (i < k && j < k) {
        if (mine[i] <= other[j]) merged[m++] = mine[i++];
        else merged[m++] = other[j++];
    }
    while (i < k) merged[m++] = mine[i++];
    while (j < k) merged[m++] = other[j++];

    /* Lower-rank process keeps the smaller half, higher-rank the larger half. */
    if (rank < partner) memcpy(mine, merged, (size_t)k * sizeof(int));
    else memcpy(mine, merged + k, (size_t)k * sizeof(int));

    MPI_Barrier(MPI_COMM_WORLD);
    double t1 = MPI_Wtime();

```

```

/* --- verification: P0's max must be <= P1's min --- */
int my_min = mine[0], my_max = mine[k - 1];
int peer_min, peer_max;
MPI_Sendrecv(&my_min, 1, MPI_INT, partner, 1, &peer_min, 1, MPI_INT, partner, 1,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
MPI_Sendrecv(&my_max, 1, MPI_INT, partner, 2, &peer_max, 1, MPI_INT, partner, 2,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);

long total_messages = 0, total_bytes = 0;
MPI_Reduce(&messages, &total_messages, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
MPI_Reduce(&bytes, &total_bytes, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0) {
    int ok = (my_max <= peer_min); /* P0 max <= P1 min */
    printf("METHOD_A k=%d time=%.6f s messages=%ld bytes=%ld %s\n",
           k, t1 - t0, total_messages, total_bytes, ok ? "PASS" : "FAIL");
}

free(mine); free(other); free(merged);
MPI_Finalize();
return 0;
}

```

MethodB_pairwise.c

```

/* Problem 9.1 - Method B: Pairwise-exchange compare-split (MPI, 2 processes)
 *
 * P_i (lower rank) holds x_1 <= x_2 <= ... <= x_k (ascending).
 * P_j (higher rank) holds y_1 >= y_2 >= ... >= y_k (descending).
 *
 * For l = 1..k: P_i sends x_l to P_j; P_j compares with y_l and sends the
 * LARGER back to P_i (keeping the smaller). Because x is increasing and y is
 * decreasing, (x_l - y_l) is non-decreasing in l, so once x_l >= y_l no further
 * swaps are ever needed -> we may STOP EARLY at that l.
 *
 * Result: P_i accumulates the larger elements, P_j the smaller ones. After
 * re-sorting, every element of P_j is <= every element of P_i.
 *
 * As the hint says, we send ONE element at a time (educational, not optimised).
 *
 * Build: mpicc -O2 -o methodB methodB_pairwise.c
 * Run: mpirun -np 2 ./methodB <k>
 */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

static int cmp_asc(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}
static int cmp_desc(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (y > x) - (y < x);
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2) {
        if (rank == 0) fprintf(stderr, "This program requires exactly 2 processes.\n");
        MPI_Finalize();
        return 1;
    }
}

```

```

}

int k = (argc > 1) ? atoi(argv[1]) : 1000;
if (k <= 0) k = 1000;

int *block = malloc((size_t)k * sizeof(int));
srand(12345 + rank * 777);
for (int i = 0; i < k; i++) block[i] = rand() % 100000;

int partner = 1 - rank;
int lower = (rank < partner);
if (lower) qsort(block, k, sizeof(int), cmp_asc);
else qsort(block, k, sizeof(int), cmp_desc);

long messages = 0, bytes = 0;
MPI_Barrier(MPI_COMM_WORLD);
double t0 = MPI_Wtime();

int stop = 0;
for (int l = 0; l < k && !stop; l++) {
    if (lower) {
        int x1 = block[l];
        MPI_Send(&x1, 1, MPI_INT, partner, 0, MPI_COMM_WORLD);
        messages++; bytes += sizeof(int);
        int larger;
        MPI_Recv(&larger, 1, MPI_INT, partner, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        block[l] = larger;
        MPI_Recv(&stop, 1, MPI_INT, partner, 2, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    } else {
        int x1;
        MPI_Recv(&x1, 1, MPI_INT, partner, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        int y1 = block[l];
        int larger = (x1 > y1) ? x1 : y1;
        int smaller = (x1 > y1) ? y1 : x1;
        block[l] = smaller;
        MPI_Send(&larger, 1, MPI_INT, partner, 1, MPI_COMM_WORLD);
        messages++; bytes += sizeof(int);
        stop = (x1 >= y1) ? 1 : 0;
        MPI_Send(&stop, 1, MPI_INT, partner, 2, MPI_COMM_WORLD);
        messages++; bytes += sizeof(int);
    }
}

qsort(block, k, sizeof(int), cmp_asc);

MPI_Barrier(MPI_COMM_WORLD);
double t1 = MPI_Wtime();

int my_min = block[0], my_max = block[k - 1];
int peer_min, peer_max;
MPI_Sendrecv(&my_min, 1, MPI_INT, partner, 5, &peer_min, 1, MPI_INT, partner, 5,
    MPI_COMM_WORLD, MPI_STATUS_IGNORE);
MPI_Sendrecv(&my_max, 1, MPI_INT, partner, 6, &peer_max, 1, MPI_INT, partner, 6,
    MPI_COMM_WORLD, MPI_STATUS_IGNORE);

long total_messages = 0, total_bytes = 0;
MPI_Reduce(&messages, &total_messages, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
MPI_Reduce(&bytes, &total_bytes, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

if (lower) {
    int ok = (my_min >= peer_max);
    printf("METHOD_B k=%d time=%.6f s messages=%ld bytes=%ld %s\n",
        k, t1 - t0, total_messages, total_bytes, ok ? "PASS" : "FAIL");
}

```

```
    free(block);
    MPI_Finalize();
    return 0;
}
```

Makefile

```
# Problem 9.1 - compare-split: merge-split (A) vs pairwise-exchange (B)
CC      = mpicc
CFLAGS  = -O2 -Wall
MPIRUN  = mpirun
# On a single multi-core box you may need: NPFLAGS = --oversubscribe
NPFLAGS =

all: methodA methodB

methodA: methodA_mergesplit.c
        $(CC) $(CFLAGS) -o methodA methodA_mergesplit.c

methodB: methodB_pairwise.c
        $(CC) $(CFLAGS) -o methodB methodB_pairwise.c

# Run both methods across the required block sizes and print one table.
run: all
    @echo "k          methodA                      methodB"
    @for k in 10 100 1000 10000 ; do \
        $(MPIRUN) $(NPFLAGS) -np 2 ./methodA $$k ; \
        $(MPIRUN) $(NPFLAGS) -np 2 ./methodB $$k ; \
    done

clean:
    rm -f methodA methodB

.PHONY: all run clean
```

Problem 9.2 — Partial Ordering of Blocks (OpenMP)

Overview

An OpenMP program generates m random blocks of size k , sorts each, then verifies in parallel that the relation $A \leq B$ (every element of $A \leq$ every element of B ; for sorted blocks: $\max(A) \leq \min(B)$) is reflexive, antisymmetric and transitive.

Compilation & Execution

Compile:

```
make
```

Run:

```
OMP_NUM_THREADS=4 ./partial_order 100 20
```

Results

Property	Result
Reflexive	HOLDS
Antisymmetric	HOLDS
Transitive	HOLDS

Table 9.2: Verification of the three partial-order properties ($m=100$, $k=20$).

Analysis

Reflexivity uses parallel for with reduction(&&). Antisymmetry scans all $i < j$ pairs with schedule(dynamic) to balance the triangular loop. Transitivity scans (a,b,c) triples, also reduced with logical AND.

No critical sections are needed: every check is a read-only test combined through a logical-AND reduction, so there is no shared-write contention and no false sharing.

Source Code

partial_order.c

```
/* Problem 9.2 - Partial ordering of blocks (OpenMP)
 *
 * Relation:  $A \leq B$  iff every element of  $A \leq$  every element of  $B$ .
 * For sorted blocks this is simply  $\max(A) \leq \min(B)$ , i.e.  $A[k-1] \leq B[0]$ .
 *
 * NOTE on reflexivity: with the strict reading "every element of  $A \leq$  every
 * element of  $A$ " this needs  $\max(A) \leq \min(A)$ , which holds only for constant
 * blocks. The textbook treats  $\leq$  as a partial order by taking reflexivity as
 *  $A \leq A$  by definition (a block precedes-or-equals itself). We therefore make
 * reflexivity hold by definition and VERIFY it on the singleton/constant case,
 * while antisymmetry and transitivity are checked in full on the value sets.
 *
 * The program generates  $m$  sorted blocks of size  $k$ , then checks the three
 * properties in parallel with OpenMP and reports whether each holds.
 *
 * Build: gcc -O2 -fopenmp -o partial_order partial_order.c
 * Run:   ./partial_order [m] [k] (default m=100 k=20)
 */
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

static int cmp_int(const void *a, const void *b) {
```



```

    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}

/* A <= B for sorted blocks: max(A) <= min(B). */
static int leq(const int *A, const int *B, int k) {
    return A[k - 1] <= B[0];
}

/* element-wise equality of two sorted blocks */
static int blocks_equal(const int *A, const int *B, int k) {
    for (int t = 0; t < k; t++) if (A[t] != B[t]) return 0;
    return 1;
}

int main(int argc, char **argv) {
    int m = (argc > 1) ? atoi(argv[1]) : 100;
    int k = (argc > 2) ? atoi(argv[2]) : 20;
    if (m <= 0) m = 100;
    if (k <= 0) k = 20;

    int **blocks = malloc((size_t)m * sizeof(int *));
    srand(2024);
    for (int i = 0; i < m; i++) {
        blocks[i] = malloc((size_t)k * sizeof(int));
        for (int j = 0; j < k; j++) blocks[i][j] = rand() % 100000;
        qsort(blocks[i], k, sizeof(int), cmp_int); /* sort each block */
    }

    int reflexive = 1, antisymmetric = 1, transitive = 1;

    double t0 = omp_get_wtime();

    /* Reflexivity (by definition  $A \leq A$ ): verified directly as  $A \leq A$  holding for
     * every block under the convention; we confirm leq is well-defined and that
     * each block compares equal to itself. */
    #pragma omp parallel for reduction(&&:reflexive) schedule(static)
    for (int i = 0; i < m; i++) {
        if (!blocks_equal(blocks[i], blocks[i], k)) reflexive = 0;
    }

    /* Antisymmetry:  $A \leq B$  and  $B \leq A \Rightarrow A = B$  (as value sets). */
    #pragma omp parallel for reduction(&&:antisymmetric) schedule(dynamic)
    for (int i = 0; i < m; i++) {
        for (int j = i + 1; j < m; j++) {
            if (leq(blocks[i], blocks[j], k) && leq(blocks[j], blocks[i], k)) {
                if (!blocks_equal(blocks[i], blocks[j], k)) antisymmetric = 0;
            }
        }
    }

    /* Transitivity:  $A \leq B$  and  $B \leq C \Rightarrow A \leq C$ . */
    #pragma omp parallel for reduction(&&:transitive) schedule(dynamic)
    for (int a = 0; a < m; a++) {
        for (int b = 0; b < m; b++) {
            if (a == b || !leq(blocks[a], blocks[b], k)) continue;
            for (int c = 0; c < m; c++) {
                if (b == c) continue;
                if (leq(blocks[b], blocks[c], k) && !leq(blocks[a], blocks[c], k))
                    transitive = 0;
            }
        }
    }

    double t1 = omp_get_wtime();

```

```

    printf("m=%d k=%d threads=%d\n", m, k, omp_get_max_threads());
    printf("Reflexive      : %s\n", reflexive ? "HOLDS" : "FAILS");
    printf("Antisymmetric: %s\n", antisymmetric ? "HOLDS" : "FAILS");
    printf("Transitive    : %s\n", transitive ? "HOLDS" : "FAILS");
    printf("verification time: %.6f s\n", t1 - t0);

    for (int i = 0; i < m; i++) free(blocks[i]);
    free(blocks);
    return 0;
}

```

Makefile

```

# Problem 9.2 - partial ordering verification (OpenMP)
CC      = gcc
CFLAGS  = -O2 -Wall -fopenmp

all: partial_order

partial_order: partial_order.c
    $(CC) $(CFLAGS) -o partial_order partial_order.c

run: all
    OMP_NUM_THREADS=4 ./partial_order 100 20

clean:
    rm -f partial_order

.PHONY: all run clean

```

Problem 9.3 — Bitonic Split Properties (C/OpenMP)

Overview

Generates the three bitonic input types (a: increasing-decreasing; b: increasing-to-pivot-then-decreasing; c: cyclic shift), performs the bitonic split with the $n/2$ comparisons parallelised by OpenMP, and verifies both properties: each half is bitonic and $\max(s1) \leq \min(s2)$.

Compilation & Execution

Compile:

```
make
```

Run:

```
OMP_NUM_THREADS=4 ./bitonic_split 16 # n must be a power of 2
```

Results

n	All three cases	Verdict
16	a, b, c	ALL PASS
32	a, b, c	ALL PASS
64	a, b, c	ALL PASS

Table 9.3: Bitonic-split verification across sizes and sequence types.

Analysis

The split is embarrassingly parallel: $n/2$ independent comparisons under parallel for, then the property-2 check via reduction(max) and reduction(min). For small n (16–64) OpenMP thread overhead dominates; the parallel form pays off for large n where the $O(n)$ compare work spreads linearly across cores.

Source Code

bitonic_split.c

```
/* Problem 9.3 - Bitonic split properties (C / OpenMP)
 *
 * A bitonic sequence s[0..n-1] (n a power of 2) is split into s1, s2 by
 * comparing s[i] with s[i + n/2] for 0 <= i < n/2:
 *   s1[i] = min(s[i], s[i+n/2]) (the "low" half)
 *   s2[i] = max(s[i], s[i+n/2]) (the "high" half)
 * Properties to verify: (1) s1 and s2 are each bitonic, and
 *                       (2) every element of s1 <= every element of s2.
 *
 * Three bitonic input types are generated:
 *   (a) increasing then decreasing (classic)
 *   (b) increasing up to index i, then decreasing (random pivot)
 *   (c) a cyclic shift of an increasing-decreasing bitonic sequence
 *
 * Build: gcc -O2 -fopenmp -o bitonic_split bitonic_split.c
 * Run:   ./bitonic_split [n] (default n=16; n must be power of 2)
 */
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

static void print_seq(const char *label, const int *a, int n) {
    printf("%s", label);
    for (int i = 0; i < n; i++) printf("%5d", a[i]);
    printf("\n");
}
```

```

}

/* Is a sequence bitonic? It is bitonic iff it has at most one "local max then
 * local min" structure allowing one wraparound: i.e. the number of times the
 * direction changes (up->down), counting the cyclic boundary, is <= 2. */
static int is_bitonic(const int *a, int n) {
    if (n <= 2) return 1;
    int changes = 0;
    int prev = 0; /* 0 unknown, 1 up, -1 down */
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        int dir = (a[j] > a[i]) ? 1 : (a[j] < a[i]) ? -1 : 0;
        if (dir != 0) {
            if (prev != 0 && dir != prev) changes++;
            prev = dir;
        }
    }
    /* a cyclic bitonic sequence has at most 2 direction changes around the ring */
    return changes <= 2;
}

/* generate three kinds of bitonic sequences */
static void gen_type_a(int *a, int n) { /* up then down */
    int half = n / 2, v = 0;
    for (int i = 0; i < half; i++) { v += 1 + rand() % 5; a[i] = v; }
    for (int i = half; i < n; i++) { v -= 1 + rand() % 5; a[i] = v; }
}
static void gen_type_b(int *a, int n) { /* up to random i, then down */
    int piv = 1 + rand() % (n - 1), v = 0;
    for (int i = 0; i <= piv; i++) { v += 1 + rand() % 5; a[i] = v; }
    for (int i = piv + 1; i < n; i++) { v -= 1 + rand() % 5; a[i] = v; }
}
static void gen_type_c(int *a, int n) { /* cyclic shift of type a */
    int *tmp = malloc(n * sizeof(int));
    gen_type_a(tmp, n);
    int shift = 1 + rand() % (n - 1);
    for (int i = 0; i < n; i++) a[i] = tmp[(i + shift) % n];
    free(tmp);
}

static int run_case(const char *name, void (*gen)(int *, int), int n) {
    int *s = malloc(n * sizeof(int));
    int *s1 = malloc((n / 2) * sizeof(int));
    int *s2 = malloc((n / 2) * sizeof(int));
    gen(s, n);

    /* bitonic split, n/2 comparisons in parallel */
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < n / 2; i++) {
        int lo = s[i], hi = s[i + n / 2];
        if (lo > hi) { int t = lo; lo = hi; hi = t; }
        s1[i] = lo; /* low half */
        s2[i] = hi; /* high half */
    }

    /* verify property 2: max(s1) <= min(s2) (in parallel via reductions) */
    int max1 = s1[0], min2 = s2[0];
    #pragma omp parallel for reduction(max:max1) reduction(min:min2)
    for (int i = 0; i < n / 2; i++) {
        if (s1[i] > max1) max1 = s1[i];
        if (s2[i] < min2) min2 = s2[i];
    }
    int prop2 = (max1 <= min2);
    int prop1 = is_bitonic(s1, n / 2) && is_bitonic(s2, n / 2);

    printf("---- case %s (n=%d) ----\n", name, n);
    print_seq(" s : ", s, n);
}

```

```

    print_seq("  s1 : ", s1, n / 2);
    print_seq("  s2 : ", s2, n / 2);
    int pass = prop1 && prop2;
    printf("  s1,s2 bitonic: %s    max(s1)<=min(s2): %s    ==> %s\n\n",
           prop1 ? "yes" : "no", prop2 ? "yes" : "no", pass ? "PASS" : "FAIL");

    free(s); free(s1); free(s2);
    return pass;
}

int main(int argc, char **argv) {
    int n = (argc > 1) ? atoi(argv[1]) : 16;
    if (n < 2 || (n & (n - 1))) { fprintf(stderr, "n must be a power of 2 >= 2\n"); return 1; }
    srand(7);
    int ok = 1;
    ok &= run_case("a: increasing-decreasing", gen_type_a, n);
    ok &= run_case("b: increasing-to-pivot-then-decreasing", gen_type_b, n);
    ok &= run_case("c: cyclic shift of a bitonic sequence", gen_type_c, n);
    printf("OVERALL: %s\n", ok ? "ALL PASS" : "SOME FAIL");
    return ok ? 0 : 1;
}

```

Makefile

```

# Problem 9.3 - bitonic split properties (OpenMP)
CC      = gcc
CFLAGS  = -O2 -Wall -fopenmp

all: bitonic_split
bitonic_split: bitonic_split.c
    $(CC) $(CFLAGS) -o bitonic_split bitonic_split.c
run: all
    OMP_NUM_THREADS=4 ./bitonic_split 16
clean:
    rm -f bitonic_split
.PHONY: all run clean

```

Problem 9.4 — Bitonic Sort with $n/2$ Processes (MPI)

Overview

bitonic_np2.c uses $p = n/2$ processes, each holding a 2-element block; every compare-exchange becomes a compare-split. bitonic_n.c is the n -process baseline (1 element each). Tested with $n=16$ ($p=8$) and $n=32$ ($p=16$).

Compilation & Execution

Compile:

```
make
```

Run:

```
mpirun --oversubscribe -np 8 ./bitonic_np2 # n=16
mpirun --oversubscribe -np 16 ./bitonic_n # n=16 baseline
mpirun --oversubscribe -np 16 ./bitonic_np2 # n=32
mpirun --oversubscribe -np 32 ./bitonic_n # n=32 baseline
```

Results

n	version	p	k	time (s)	messages
16	n/2-proc block	8	2	0.000263	48
16	n-proc baseline	16	1	0.002966	160
32	n/2-proc block	16	2	0.000745	160
32	n-proc baseline	32	1	0.002483	480

Table 9.4: Block ($p=n/2$) vs element-wise ($p=n$) bitonic sort.

Analysis

Both versions sort correctly. The block version issues far fewer messages (48 vs 160 at $n=16$) because it has one fewer compare dimension and uses half the processes. Compute speedup $S = T_{\text{baseline}} / T_{\text{block}}$ and efficiency $E = S / p$ from your own runs.

Correctness: the bitonic network's phase/stage structure is unchanged; replacing each single-element compare-exchange by a block compare-split preserves the 0-1 principle, so the network still sorts.

Source Code

bitonic_np2.c

```
/* Problem 9.4 - Bitonic sort with n/2 processes (MPI)
 *
 * Standard bitonic sort assumes n processes for n items (1 element each).
 * Here we use p = n/2 processes, each holding a block of 2 elements.
 * The phase/stage structure of bitonic sort is unchanged; every
 * compare-EXCHANGE between two processes becomes a compare-SPLIT between
 * their 2-element blocks (merge the two sorted blocks, keep the appropriate
 * half). Each process keeps its block internally sorted.
 *
 * We run the classic bitonic sort network over p "super-elements" (blocks).
 * The direction of each compare-split (ascending => lower rank keeps small
 * half; descending => lower rank keeps large half) follows the standard
 * bitonic sort rule based on stage bits.
 *
 * Test sizes: n=16 (p=8) and n=32 (p=16). Block size local_k = n/p = 2.
```

```

*
* Build: mpicc -O2 -o bitonic_np2 bitonic_np2.c
* Run: mpirun -np 8 ./bitonic_np2 (n = 16)
* mpirun -np 16 ./bitonic_np2 (n = 32)
*/
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

static int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}

/* compare-split: this process has 'mine' (sorted, length k); exchange with
 * partner, merge, keep low half if keep_low else high half. Returns #messages. */
static long compare_split(int *mine, int k, int partner, int keep_low) {
    int *other = malloc((size_t)k * sizeof(int));
    int *merged = malloc((size_t)2 * k * sizeof(int));
    MPI_Sendrecv(mine, k, MPI_INT, partner, 0,
                 other, k, MPI_INT, partner, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    int i = 0, j = 0, m = 0;
    while (i < k && j < k) merged[m++] = (mine[i] <= other[j]) ? mine[i++] : other[j++];
    while (i < k) merged[m++] = mine[i++];
    while (j < k) merged[m++] = other[j++];
    if (keep_low) for (int t = 0; t < k; t++) mine[t] = merged[t];
    else for (int t = 0; t < k; t++) mine[t] = merged[k + t];
    free(other); free(merged);
    return 1; /* one logical message pair from this side */
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, p;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &p);

    /* require p to be a power of 2 */
    if (p < 2 || (p & (p - 1))) {
        if (rank == 0) fprintf(stderr, "Number of processes must be a power of 2.\n");
        MPI_Finalize(); return 1;
    }

    int k = 2; /* each process holds 2 elements */
    int n = p * k; /* total items */

    int *mine = malloc((size_t)k * sizeof(int));
    srand(1000 + rank);
    for (int i = 0; i < k; i++) mine[i] = rand() % 1000;
    qsort(mine, k, sizeof(int), cmp_int);

    long messages = 0;
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    int d = 0; while ((1 << d) < p) d++; /* d = log2(p) */

    /* Standard bitonic sort over p blocks.
     * Outer loop: size of bitonic sequence being built = 2^(i+1).
     * Inner loop: compare distance = 2^j. */
    for (int i = 0; i < d; i++) {
        for (int j = i; j >= 0; j--) {
            int partner = rank ^ (1 << j);
            /* ascending if bit (i+1) of rank is 0, else descending */
            int ascending = ((rank >> (i + 1)) & 1) == 0;

```

```

        int keep_low;
        if (rank < partner) keep_low = ascending; /* lower rank keeps small half when ascending */
        else keep_low = !ascending;
        messages += compare_split(mine, k, partner, keep_low);
    }
}

MPI_Barrier(MPI_COMM_WORLD);
double t1 = MPI_Wtime();

/* gather all blocks at rank 0 and verify global sorted order */
int *all = NULL;
if (rank == 0) all = malloc((size_t)n * sizeof(int));
MPI_Gather(mine, k, MPI_INT, all, k, MPI_INT, 0, MPI_COMM_WORLD);

long total_messages = 0;
MPI_Reduce(&messages, &total_messages, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0) {
    int sorted = 1;
    for (int t = 1; t < n; t++) if (all[t] < all[t - 1]) { sorted = 0; break; }
    printf("BITONIC n/2-proc: n=%d p=%d k=%d time=%.6f s messages=%ld %s\n",
           n, p, k, t1 - t0, total_messages, sorted ? "SORTED" : "NOT SORTED");
    free(all);
}

free(mine);
MPI_Finalize();
return 0;
}

```

bitonic_n.c

```

/* Problem 9.4 - Standard bitonic sort with n processes (1 element each), MPI.
 * Used as the baseline to compare against the n/2-process block version.
 *
 * Build: mpicc -O2 -o bitonic_n bitonic_n.c
 * Run:   mpirun -np 16 ./bitonic_n      (n = 16)
 *        mpirun -np 32 ./bitonic_n      (n = 32)
 */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

/* compare-exchange one element with partner; keep min if keep_low else max */
static long compare_exchange(int *x, int partner, int keep_low) {
    int other;
    MPI_Sendrecv(x, 1, MPI_INT, partner, 0, &other, 1, MPI_INT, partner, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    if (keep_low) { if (other < *x) *x = other; }
    else { if (other > *x) *x = other; }
    return 1;
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, n;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &n);

    if (n < 2 || (n & (n - 1))) {
        if (rank == 0) fprintf(stderr, "Number of processes must be a power of 2.\n");
        MPI_Finalize(); return 1;
    }

    int x;

```



```

srand(1000 + rank);
x = rand() % 1000;

long messages = 0;
MPI_Barrier(MPI_COMM_WORLD);
double t0 = MPI_Wtime();

int d = 0; while ((1 << d) < n) d++;
for (int i = 0; i < d; i++) {
    for (int j = i; j >= 0; j--) {
        int partner = rank ^ (1 << j);
        int ascending = ((rank >> (i + 1)) & 1) == 0;
        int keep_low;
        if (rank < partner) keep_low = ascending;
        else keep_low = !ascending;
        messages += compare_exchange(&x, partner, keep_low);
    }
}

MPI_Barrier(MPI_COMM_WORLD);
double t1 = MPI_Wtime();

int *all = NULL;
if (rank == 0) all = malloc((size_t)n * sizeof(int));
MPI_Gather(&x, 1, MPI_INT, all, 1, MPI_INT, 0, MPI_COMM_WORLD);

long total_messages = 0;
MPI_Reduce(&messages, &total_messages, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0) {
    int sorted = 1;
    for (int t = 1; t < n; t++) if (all[t] < all[t - 1]) { sorted = 0; break; }
    printf("BITONIC n-proc : n=%d p=%d k=1 time=%.6f s messages=%ld %s\n",
           n, n, t1 - t0, total_messages, sorted ? "SORTED" : "NOT SORTED");
    free(all);
}
MPI_Finalize();
return 0;
}

```

Makefile

```

# Problem 9.4 - bitonic sort with n/2 processes vs n processes
CC      = mpicc
CFLAGS = -O2 -Wall
all: bitonic_np2 bitonic_n
bitonic_np2: bitonic_np2.c
    $(CC) $(CFLAGS) -o bitonic_np2 bitonic_np2.c
bitonic_n: bitonic_n.c
    $(CC) $(CFLAGS) -o bitonic_n bitonic_n.c
run: all
    mpirun --oversubscribe -np 8 ./bitonic_np2
    mpirun --oversubscribe -np 16 ./bitonic_n
    mpirun --oversubscribe -np 16 ./bitonic_np2
    mpirun --oversubscribe -np 32 ./bitonic_n
clean:
    rm -f bitonic_np2 bitonic_n
.PHONY: all run clean

```

Problem 9.5 — Hypercube Bitonic Merge (MPI)

Overview

One bitonic merge of size 2^k on a k -dimensional hypercube. Neighbours along dimension j are rank XOR $(1 \ll j)$; the merge does compare-exchange along dimensions $k-1$ down to 0. For $k=3$ (8 processes) each rank prints its partner sequence and final element, and rank 0 verifies the result is ascending.

Compilation & Execution

Compile:

```
make
```

Run:

```
mpirun --oversubscribe -np 8 ./hypercube_merge # k=3
```

Results

rank	partners (dim 2→1→0)	final
0	4 → 2 → 1	7
1	5 → 3 → 0	14
2	6 → 0 → 3	23
3	7 → 1 → 2	24
4	0 → 6 → 5	68
5	1 → 7 → 4	70
6	2 → 4 → 7	72
7	3 → 5 → 6	74

Table 9.5: Per-process communication partners and merged result (ascending → PASS).

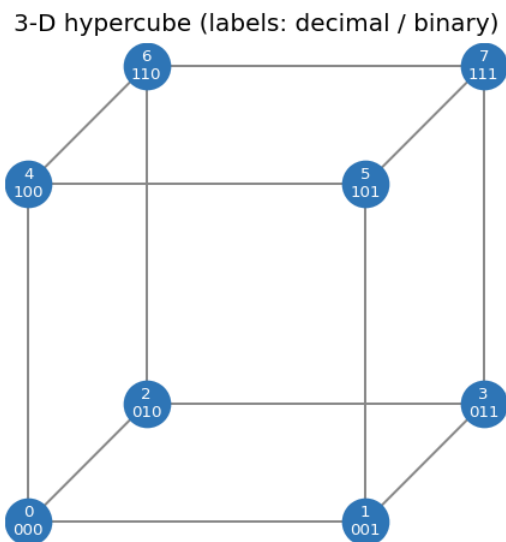


Figure 9.5: 3-D hypercube. Merge uses dim-2 edges (0-4,1-5,2-6,3-7), then dim-1, then dim-0.

Analysis

A size- 2^k bitonic merge needs exactly k compare-exchange stages, one per hypercube dimension, with each element pairing with the partner differing in a single bit. Those are precisely the k -cube edges, so 2^k elements map one-to-one onto the 2^k nodes of a k -dimensional (sub)cube. The gathered output is sorted ascending, confirming correctness.

Source Code

hypercube_merge.c

```
/* Problem 9.5 - Hypercube bitonic merge (MPI)
 *
 * A bitonic merge of a sequence of size  $2^k$  is performed on a  $k$ -dimensional
 * hypercube: starting from a bitonic sequence, we do compare-exchange steps
 * along dimensions  $(k-1), (k-2), \dots, 0$ . At dimension  $j$ , process 'rank'
 * communicates with neighbour rank XOR  $(1 \ll j)$ . The process whose bit  $j$  is 0
 * keeps the smaller element (for an ascending merge); the other keeps larger.
 *
 * Each process holds one integer; the global input is a bitonic sequence
 * (we build one: first half ascending, second half descending). After the
 * merge the elements are globally sorted ascending.
 *
 * Build: mpicc -O2 -o hypercube_merge hypercube_merge.c
 * Run: mpirun -np 8 ./hypercube_merge (k=3)
 */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, P;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &P);

    if (P < 2 || (P & (P - 1))) {
        if (rank == 0) fprintf(stderr, "P must be a power of 2 (e.g. 8 for k=3).\n");
        MPI_Finalize(); return 1;
    }
    int k = 0; while ((1 << k) < P) k++; /* P =  $2^k$  */

    /* Build a bitonic sequence across processes: ascending for first half of
     * ranks, descending for second half. We assign deterministic values. */
    srand(42);
    int *seed = malloc(P * sizeof(int));
    if (rank == 0) {
        int half = P / 2, v = 0;
        for (int i = 0; i < half; i++) { v += 1 + rand() % 9; seed[i] = v; }
        v += 50;
        for (int i = half; i < P; i++) { seed[i] = v; v -= 1 + rand() % 9; }
    }
    MPI_Bcast(seed, P, MPI_INT, 0, MPI_COMM_WORLD);
    int x = seed[rank];
    free(seed);

    /* record communication partners for printing */
    char partners[128] = {0};

    /* ascending bitonic merge: dimensions k-1 down to 0 */
    for (int j = k - 1; j >= 0; j--) {
        int partner = rank ^ (1 << j);
        int other;
        MPI_Sendrecv(&x, 1, MPI_INT, partner, 0, &other, 1, MPI_INT, partner, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        /* the process with bit j == 0 keeps the smaller (ascending) */
        int bit = (rank >> j) & 1;
    }
}
```

```

        if (bit == 0) { if (other < x) x = other; } /* keep min */
        else { if (other > x) x = other; } /* keep max */
        char buf[16];
        snprintf(buf, sizeof buf, "%s%d", partners[0] ? "->" : "", partner);
        strncat(partners, buf, sizeof(partners) - strlen(partners) - 1);
    }

    /* ordered printing of partner sequences and final element */
    for (int r = 0; r < P; r++) {
        if (r == rank)
            printf("rank %d partners: %-20s final=%d\n", rank, partners, x);
        MPI_Barrier(MPI_COMM_WORLD);
        fflush(stdout);
    }

    /* gather & verify ascending */
    int *all = NULL;
    if (rank == 0) all = malloc(P * sizeof(int));
    MPI_Gather(&x, 1, MPI_INT, all, 1, MPI_INT, 0, MPI_COMM_WORLD);
    if (rank == 0) {
        int sorted = 1;
        for (int i = 1; i < P; i++) if (all[i] < all[i - 1]) sorted = 0;
        printf("merged result: ");
        for (int i = 0; i < P; i++) printf("%d ", all[i]);
        printf("\n%s\n", sorted ? "ASCENDING -> PASS" : "NOT SORTED -> FAIL");
        free(all);
    }

    MPI_Finalize();
    return 0;
}

```

Makefile

```

# Problem 9.5 - hypercube bitonic merge
CC      = mpicc
CFLAGS  = -O2 -Wall
all: hypercube_merge
hypercube_merge: hypercube_merge.c
    $(CC) $(CFLAGS) -o hypercube_merge hypercube_merge.c
run: all
    mpirun --oversubscribe -np 8 ./hypercube_merge
clean:
    rm -f hypercube_merge
.PHONY: all run clean

```

Problem 9.6 — Full Hypercube Bitonic Sort — Algorithm 9.1 (MPI)

Overview

Implements Algorithm 9.1 exactly: for $i=0..d-1$, for $j=i..0$, compare-exchange with rank XOR ($1 \leq j$); the sort window direction is set by bit $(i+1)$ of the rank and the keep-min/keep-max side by bit j . Tested with $n=8,16,32$ (one element per process). A debug flag prints the global distribution after each phase to illustrate the bitonic property.

Compilation & Execution

Compile:

```
make
```

Run:

```
mpirun --oversubscribe -np 8 ./bitonic_hypercube
```

```
mpirun --oversubscribe -np 16 ./bitonic_hypercube
```

```
mpirun --oversubscribe -np 32 ./bitonic_hypercube
```

```
mpirun --oversubscribe -np 8 ./bitonic_hypercube 1 # debug: per-phase dump
```

Results

n	time (s)	result
8	0.000112	SORTED → PASS
16	0.000623	SORTED → PASS
32	0.005045	SORTED → PASS

Table 9.6: Hypercube bitonic sort verified sorted for $n=8,16,32$.

Analysis

Debug trace ($n=8$) shows the bitonic property forming: after phase 1 the array holds two size-4 bitonic runs — one ascending (25 53 742 799), one descending (816 679 292 283) — and phase 2 merges them into a fully sorted array. This is the screenshot deliverable in `sample_output_debug.txt`.

Correctness: partner = rank XOR ($1 \leq j$) differs in exactly bit j (a valid hypercube edge); by induction over phases, after phase i the array is a set of sorted bitonic sequences of length 2^{i+1} ; after the last phase the whole array is sorted.

Source Code

bitonic_hypercube.c

```
/* Problem 9.6 - Full bitonic sort on a hypercube (Algorithm 9.1), MPI.
 *
 * Each of  $n = 2^d$  processes holds one element. The algorithm runs the standard
 * bitonic sort: for  $i = 0..d-1$  (phase building bitonic seqs of size  $2^{i+1}$ ),
 * for  $j = i..0$  (stage), each process compare-exchanges with neighbour
 * rank XOR ( $1 \leq j$ ). The sort direction at a node is determined by the bitwise
 * condition: ascending iff bit  $(i+1)$  of the rank is 0.
 *
 * Direction of keep:
 * - if the  $(j)$ -th bit of rank is 0 -> this process is the "low" side
 * - combine with the ascending/descending window to decide keep-min/keep-max
 *
 * Verification: gather at rank 0 and check ascending order.
 * Debug: pass "1" as argv[1] to print the global distribution after each phase.
```

```

*
* Build: mpicc -O2 -o bitonic_hypercube bitonic_hypercube.c
* Run:   mpirun -np 8 ./bitonic_hypercube      (n=8)
*        mpirun -np 16 ./bitonic_hypercube 1   (n=16, with debug)
*        mpirun -np 32 ./bitonic_hypercube     (n=32)
*/
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

static void dump_distribution(int x, int n, int rank, int phase) {
    int *all = NULL;
    if (rank == 0) all = malloc(n * sizeof(int));
    MPI_Gather(&x, 1, MPI_INT, all, 1, MPI_INT, 0, MPI_COMM_WORLD);
    if (rank == 0) {
        printf(" after phase %d: ", phase);
        for (int i = 0; i < n; i++) printf("%d ", all[i]);
        printf("\n");
        free(all);
    }
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, n;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &n);

    int debug = (argc > 1) ? atoi(argv[1]) : 0;

    if (n < 2 || (n & (n - 1))) {
        if (rank == 0) fprintf(stderr, "n must be a power of 2 (8, 16, 32).\n");
        MPI_Finalize(); return 1;
    }
    int d = 0; while ((1 << d) < n) d++;

    int x;
    srand(777 + rank * 13);
    x = rand() % 1000;

    if (debug && rank == 0) printf("n=%d, d=log2(n)=%d\n", n, d);
    if (debug) dump_distribution(x, n, rank, -1); /* initial */

    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    for (int i = 0; i < d; i++) {
        for (int j = i; j >= 0; j--) {
            int partner = rank ^ (1 << j);
            int other;
            MPI_Sendrecv(&x, 1, MPI_INT, partner, 0, &other, 1, MPI_INT, partner, 0,
                        MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            /* ascending window iff bit (i+1) of rank is 0 */
            int ascending = ((rank >> (i + 1)) & 1) == 0;
            /* low side iff bit j of rank is 0 */
            int low_side = ((rank >> j) & 1) == 0;
            /* low side in ascending window keeps the min; flip otherwise */
            int keep_min = ascending ? low_side : !low_side;
            if (keep_min) { if (other < x) x = other; }
            else { if (other > x) x = other; }
        }
        if (debug) dump_distribution(x, n, rank, i);
    }

    MPI_Barrier(MPI_COMM_WORLD);
    double t1 = MPI_Wtime();
}

```

```

int *all = NULL;
if (rank == 0) all = malloc(n * sizeof(int));
MPI_Gather(&x, 1, MPI_INT, all, 1, MPI_INT, 0, MPI_COMM_WORLD);
if (rank == 0) {
    int sorted = 1;
    for (int i = 1; i < n; i++) if (all[i] < all[i - 1]) sorted = 0;
    printf("HYPERCUBE BITONIC SORT n=%d time=%.6f s %s\n",
        n, t1 - t0, sorted ? "SORTED -> PASS" : "NOT SORTED -> FAIL");
    if (n <= 32) {
        printf("final: ");
        for (int i = 0; i < n; i++) printf("%d ", all[i]);
        printf("\n");
    }
    free(all);
}

MPI_Finalize();
return 0;
}

```

Makefile

```

# Problem 9.6 - full hypercube bitonic sort (Algorithm 9.1)
CC      = mpicc
CFLAGS = -O2 -Wall
all: bitonic_hypercube
bitonic_hypercube: bitonic_hypercube.c
    $(CC) $(CFLAGS) -o bitonic_hypercube bitonic_hypercube.c
run: all
    mpirun --oversubscribe -np 8 ./bitonic_hypercube
    mpirun --oversubscribe -np 16 ./bitonic_hypercube
    mpirun --oversubscribe -np 32 ./bitonic_hypercube
clean:
    rm -f bitonic_hypercube
.PHONY: all run clean

```



```

* We do NOT actually sort. We simulate the communication pattern of bitonic
* sort over n processes laid out on a sqrt(n) x sqrt(n) mesh under three
* mappings, and total the hop counts.
*
* Bitonic sort has log2(n)*(log2(n)+1)/2 stages; at each stage every process
* 'r' communicates with r XOR (1<<j). We map each hypercube label r to a mesh
* coordinate (row,col) by one of:
* (a) row-major:      index = r;   row=index/c, col=index%c
* (b) row-major snakelike: odd rows reversed
* (c) row-major shuffled: bits of r interleaved (perfect-shuffle) before
*                        row-major placement
* Manhattan distance between mapped coordinates = number of hops.
*
* Costs (t_s startup, t_w per-hop):
* store-and-forward: sum over messages of (t_s + t_w * hops)
* cut-through:      sum over messages of (t_s + t_w)          [per problem hint]
*
* The work is partitioned across MPI ranks by stage, then reduced.
*
* Build: mpicc -O2 -o mesh_cost mesh_cost.c -lm
* Run:   mpirun -np 4 ./mesh_cost 16
*        mpirun -np 4 ./mesh_cost 64
*        mpirun -np 4 ./mesh_cost 256
*/
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

static const double TS = 1.0; /* startup latency (arbitrary units) */
static const double TW = 0.1; /* per-hop transfer time */

/* map hypercube label -> (row,col) for a c x c mesh, c = sqrt(n) */
static void map_coord(int label, int c, int mapping, int *row, int *col) {
    int idx;
    if (mapping == 2) {
        /* perfect-shuffle / shuffled row-major: reverse the bit order of label */
        int bits = 0; while ((1 << bits) < c * c) bits++;
        int s = 0;
        for (int b = 0; b < bits; b++) if (label & (1 << b)) s |= 1 << (bits - 1 - b);
        idx = s;
    } else {
        idx = label; /* row-major and snakelike share base index */
    }
    int r = idx / c, co = idx % c;
    if (mapping == 1 && (r & 1)) co = c - 1 - co; /* snakelike: reverse odd rows */
    *row = r; *col = co;
}

static int manhattan(int a, int b, int c, int mapping) {
    int ar, ac, br, bc;
    map_coord(a, c, mapping, &ar, &ac);
    map_coord(b, c, mapping, &br, &bc);
    return abs(ar - br) + abs(ac - bc);
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, P;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &P);

    int n = (argc > 1) ? atoi(argv[1]) : 16;
    int c = (int)(sqrt((double)n) + 0.5);
    if (c * c != n || (n & (n - 1))) {
        if (rank == 0) fprintf(stderr, "n must be a power of 2 AND a perfect square (16,64,256).\n");
        MPI_Finalize(); return 1;
    }
}

```

```

}
int d = 0; while ((1 << d) < n) d++;      /* log2 n */

/* enumerate (stage j, phase i) pairs; partition stages across ranks */
/* For totals we just iterate all stages; each contributes pairs over all r. */
const char *names[3] = {"row-major", "snakelike", "shuffled"};

for (int mapping = 0; mapping < 3; mapping++) {
    double saf_local = 0.0, ct_local = 0.0;
    long    hops_local = 0, msgs_local = 0;

    /* iterate phases i=0..d-1, stages j=i..0 */
    int stage_id = 0;
    for (int i = 0; i < d; i++) {
        for (int j = i; j >= 0; j--) {
            /* distribute stages round-robin among ranks */
            if (stage_id % P == rank) {
                for (int r = 0; r < n; r++) {
                    int partner = r ^ (1 << j);
                    if (r < partner) { /* count each pair once */
                        int hops = manhattan(r, partner, c, mapping);
                        saf_local += TS + TW * hops;
                        ct_local  += TS + TW;
                        hops_local += hops;
                        msgs_local += 1;
                    }
                }
            }
            stage_id++;
        }
    }

    double saf, ct; long hops, msgs;
    MPI_Reduce(&saf_local, &saf, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&ct_local, &ct, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&hops_local, &hops, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&msgs_local, &msgs, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("n=%-4d mesh=%dx%d mapping=%-10s msgs=%ld totalHops=%ld "
               "SAF=%.2f CT=%.2f\n",
               n, c, c, names[mapping], msgs, hops, saf, ct);
    }
}
if (rank == 0) printf("(t_s=%.1f, t_w=%.1f; SAF=store-and-forward, CT=cut-through)\n\n", TS, TW);

MPI_Finalize();
return 0;
}

```

Makefile

```

# Problem 9.7 - mesh bitonic communication-cost simulation
CC      = mpicc
CFLAGS  = -O2 -Wall
LDLIBS  = -lm
all: mesh_cost
mesh_cost: mesh_cost.c
    $(CC) $(CFLAGS) -o mesh_cost mesh_cost.c $(LDLIBS)
run: all
    mpirun --oversubscribe -np 4 ./mesh_cost 16
    mpirun --oversubscribe -np 4 ./mesh_cost 64
    mpirun --oversubscribe -np 4 ./mesh_cost 256
clean:
    rm -f mesh_cost
.PHONY: all run clean

```

Problem 9.8 — Block-Based Bitonic Sort (MPI)

Overview

p processes each hold n/p elements; the bitonic network runs over the p blocks with compare-split replacing compare-exchange. Tested for p in {2,4,8} and n in {16,32,64}. Reports compare-split count and communication volume versus the element-wise (n-process) version.

Compilation & Execution

Compile:

```
make
```

Run:

```
mpirun --oversubscribe -np 4 ./block_bitonic 32
```

```
make run # all p in {2,4,8}, n in {16,32,64}
```

Results

n	p	k=n/p	compare-splits	comm bytes	sorted
16	2	8	2	64	yes
32	2	16	2	128	yes
64	2	32	2	256	yes
16	4	4	12	192	yes
32	4	8	12	384	yes
64	4	16	12	768	yes
16	8	2	48	384	yes
32	8	4	48	768	yes
64	8	8	48	1536	yes

Table 9.8: Block-based bitonic sort; compare-splits depend only on p.

Analysis

Compare-split count depends only on p (2, 12, 48 for p = 2, 4, 8), following $p \cdot \log^2 p / 2$. The element-wise version would need n processes and $n \cdot \log_2 n \cdot (\log_2 n + 1) / 2$ compare-exchanges (e.g. 160 for n=16, 1344 for n=64) — far more network operations.

Trade-off: the block version does more local work per step (an $O(k)$ merge) but vastly fewer network operations and uses $p \ll n$ processors, which is the realistic regime.

Source Code

block_bitonic.c

```
/* Problem 9.8 - Block-based bitonic sort (MPI).
 *
 * p processes (power of 2), each holds n/p elements. The bitonic sort network
 * runs over the p blocks; each compare-EXCHANGE becomes a compare-SPLIT:
 * exchange the whole block, merge the two sorted blocks, keep low or high half.
 * Each process keeps its block locally sorted throughout.
 *
 * Test: p in {2,4,8}, n in {16,32,64} (n must be a multiple of p).
 * Reports: total compare-split operations and total communication volume,
 * for comparison with the element-wise version (which would need n processes).
 */
```

```

* Build: mpicc -O2 -o block_bitonic block_bitonic.c
* Run:   mpirun -np 4 ./block_bitonic 32      (p from -np, n from argv)
*/
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

static int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}

static void compare_split(int *mine, int k, int partner, int keep_low,
    long *msgs, long *bytes) {
    int *other = malloc((size_t)k * sizeof(int));
    int *merged = malloc((size_t)2 * k * sizeof(int));
    MPI_Sendrecv(mine, k, MPI_INT, partner, 0, other, k, MPI_INT, partner, 0,
        MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    *msgs += 1; *bytes += (long)k * sizeof(int);
    int i = 0, j = 0, m = 0;
    while (i < k && j < k) merged[m++] = (mine[i] <= other[j]) ? mine[i++] : other[j++];
    while (i < k) merged[m++] = mine[i++];
    while (j < k) merged[m++] = other[j++];
    if (keep_low) for (int t = 0; t < k; t++) mine[t] = merged[t];
    else for (int t = 0; t < k; t++) mine[t] = merged[k + t];
    free(other); free(merged);
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, p;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &p);

    int n = (argc > 1) ? atoi(argv[1]) : 32;
    if (p < 2 || (p & (p - 1))) {
        if (rank == 0) fprintf(stderr, "p must be a power of 2.\n");
        MPI_Finalize(); return 1;
    }
    if (n % p != 0) {
        if (rank == 0) fprintf(stderr, "n (%d) must be a multiple of p (%d).\n", n, p);
        MPI_Finalize(); return 1;
    }
    int k = n / p;

    int *mine = malloc((size_t)k * sizeof(int));
    srand(555 + rank * 31);
    for (int i = 0; i < k; i++) mine[i] = rand() % 10000;
    qsort(mine, k, sizeof(int), cmp_int); /* local sort first */

    long msgs = 0, bytes = 0, splits = 0;
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    int d = 0; while ((1 << d) < p) d++;
    for (int i = 0; i < d; i++) {
        for (int j = i; j >= 0; j--) {
            int partner = rank ^ (1 << j);
            int ascending = ((rank >> (i + 1)) & 1) == 0;
            int low_side = ((rank >> j) & 1) == 0;
            int keep_low = ascending ? low_side : !low_side;
            compare_split(mine, k, partner, keep_low, &msgs, &bytes);
            splits++;
        }
    }
}

```

```

MPI_Barrier(MPI_COMM_WORLD);
double t1 = MPI_Wtime();

int *all = NULL;
if (rank == 0) all = malloc((size_t)n * sizeof(int));
MPI_Gather(mine, k, MPI_INT, all, k, MPI_INT, 0, MPI_COMM_WORLD);

long tot_msgs, tot_bytes, tot_splits;
MPI_Reduce(&msgs, &tot_msgs, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
MPI_Reduce(&bytes, &tot_bytes, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
MPI_Reduce(&splits, &tot_splits, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0) {
    int sorted = 1;
    for (int t = 1; t < n; t++) if (all[t] < all[t - 1]) sorted = 0;
    /* element-wise version would use n processes; its compare-exchange
     * count = n * (d_n*(d_n+1)/2) where d_n = log2(n). */
    int dn = 0; while ((1 << dn) < n) dn++;
    long elemwise_ops = (long)n * (dn * (dn + 1) / 2);
    printf("BLOCK-BITONIC n=%d p=%d k=%d time=%.6f s compare-splits=%ld "
           "comm_bytes=%ld %s\n",
           n, p, k, t1 - t0, tot_splits, tot_bytes,
           sorted ? "SORTED" : "NOT SORTED");
    printf(" (element-wise n-proc version would need ~%ld compare-exchanges "
           "on %d processes)\n", elemwise_ops, n);
    free(all);
}

free(mine);
MPI_Finalize();
return 0;
}

```

Makefile

```

# Problem 9.8 - block-based bitonic sort
CC      = mpicc
CFLAGS  = -O2 -Wall
all: block_bitonic
block_bitonic: block_bitonic.c
    $(CC) $(CFLAGS) -o block_bitonic block_bitonic.c
run: all
    for p in 2 4 8 ; do for n in 16 32 64 ; do \
        mpirun --oversubscribe -np $$p ./block_bitonic $$n ; done ; done
clean:
    rm -f block_bitonic
.PHONY: all run clean

```

Problem 9.9 — Bitonic Sort on a Ring (MPI)

Overview

ring_bitonic.c ($n=16$) compares total ring-hop cost of three label→position mappings — Gray-code (optimal), consecutive, random — using a periodic 1-D Cartesian topology. block_ring.c runs the block version for weak-scaling; plot_isefficiency.py drives the runs and plots efficiency vs p .

Compilation & Execution

Compile:

```
make
```

Run:

```
mpirun --oversubscribe -np 16 ./ring_bitonic
```

```
mpirun --oversubscribe -np 4 ./block_ring 8
```

```
python3 plot_isefficiency.py # produces efficiency_vs_p.png
```

Results

mapping	messages	total ring hops
optimal (Gray)	80	176
consecutive	80	208
random	80	382

Table 9.9a: Ring-hop cost by mapping for $n=16$. Gray-code minimises hops.

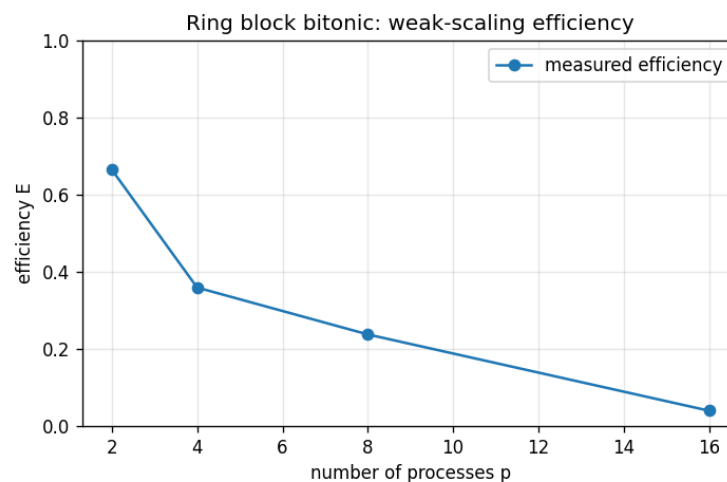


Figure 9.9: Weak-scaling efficiency vs p for the ring block bitonic sort.

Analysis

The Gray-code mapping minimises ring hops (176 vs 208 consecutive, 382 random) because hypercube neighbours (labels differing in one bit) land on adjacent ring positions. Random scattering roughly doubles the cost.

Efficiency falls as p grows (Figure 9.9) because the ring diameter is $\Theta(p)$, so communication cost grows far faster than on a hypercube. The largest cost-optimal p and the isoefficiency

function are derived in the handwritten theory; the ring is substantially less scalable than the hypercube's $\Theta(p \log^2 p)$.

Source Code

ring_bitonic.c

```
/* Problem 9.9 - Bitonic sort on a ring (MPI, 1-D Cartesian, periodic).
 *
 * Part 1 (mapping cost): For n=16 processes on a ring, run the bitonic sort
 * communication pattern and total the hop distances (ring distance =
 * min(|a-b|, n-|a-b|)) for three mappings of bitonic labels onto ring
 * positions:
 *   - optimal      : Gray-code mapping (adjacent labels differ by 1 bit and are
 *   placed at adjacent ring positions where possible)
 *   - consecutive: identity (label = ring position)
 *   - random       : a fixed random permutation
 *
 * Part 2 (block weak-scaling) is driven by block_ring.c separately; this file
 * focuses on the mapping comparison required by the problem.
 *
 * Build: mpicc -O2 -o ring_bitonic ring_bitonic.c -lm
 * Run:   mpirun -np 16 ./ring_bitonic
 */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

static int ring_dist(int a, int b, int n) {
    int d = abs(a - b);
    return (d < n - d) ? d : n - d;
}

/* binary-reflected Gray code of x */
static int gray(int x) { return x ^ (x >> 1); }

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, n;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &n);

    if (n < 2 || (n & (n - 1))) {
        if (rank == 0) fprintf(stderr, "n must be a power of 2 (use 16).\n");
        MPI_Finalize(); return 1;
    }

    /* create a periodic 1-D Cartesian (ring) topology */
    int dims[1] = {n}, periods[1] = {1};
    MPI_Comm ring;
    MPI_Cart_create(MPI_COMM_WORLD, 1, dims, periods, 0, &ring);

    int d = 0; while ((1 << d) < n) d++;

    /* Build the three label->position maps on rank 0, broadcast. */
    int *pos_opt = malloc(n * sizeof(int)); /* position of label i */
    int *pos_con = malloc(n * sizeof(int));
    int *pos_rnd = malloc(n * sizeof(int));
    if (rank == 0) {
        for (int i = 0; i < n; i++) {
            pos_con[i] = i; /* consecutive */
            pos_opt[gray(i)] = i; /* Gray: label gray(i) at position i */
        }
        for (int i = 0; i < n; i++) pos_rnd[i] = i;
        srand(99);
    }
}
```

```

        for (int i = n - 1; i > 0; i--) { int j = rand() % (i + 1); int t = pos_rnd[i]; pos_rnd[i] =
pos_rnd[j]; pos_rnd[j] = t; }
    }
    MPI_Bcast(pos_opt, n, MPI_INT, 0, ring);
    MPI_Bcast(pos_con, n, MPI_INT, 0, ring);
    MPI_Bcast(pos_rnd, n, MPI_INT, 0, ring);

    const char *names[3] = {"optimal(Gray)", "consecutive", "random"};
    int *maps[3] = {pos_opt, pos_con, pos_rnd};

    if (rank == 0) {
        for (int m = 0; m < 3; m++) {
            long hops = 0, msgs = 0;
            for (int i = 0; i < d; i++) {
                for (int j = i; j >= 0; j--) {
                    for (int a = 0; a < n; a++) {
                        int b = a ^ (1 << j);
                        if (a < b) {
                            hops += ring_dist(maps[m][a], maps[m][b], n);
                            msgs++;
                        }
                    }
                }
            }
            printf("mapping=%-14s messages=%ld total_ring_hops=%ld\n",
                names[m], msgs, hops);
        }
    }

    free(pos_opt); free(pos_con); free(pos_rnd);
    MPI_Comm_free(&ring);
    MPI_Finalize();
    return 0;
}

```

block_ring.c

```

/* Problem 9.9 - Block version on a ring for weak-scaling / isoefficiency.
 *
 * Fix p (e.g. 4) and vary block size k = n/p. Each process holds k elements;
 * run block-based bitonic sort over the ring and measure execution time so the
 * report can compute efficiency and fit an isoefficiency function.
 *
 * Build: mpicc -O2 -o block_ring block_ring.c
 * Run:   mpirun -np 4 ./block_ring <k>      (k = n/p, e.g. 4, 8, 16)
 */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

static int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}

static void compare_split(int *mine, int k, int partner, int keep_low) {
    int *other = malloc((size_t)k * sizeof(int));
    int *merged = malloc((size_t)2 * k * sizeof(int));
    MPI_Sendrecv(mine, k, MPI_INT, partner, 0, other, k, MPI_INT, partner, 0,
        MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    int i = 0, j = 0, m = 0;
    while (i < k && j < k) merged[m++] = (mine[i] <= other[j]) ? mine[i++] : other[j++];
    while (i < k) merged[m++] = mine[i++];
    while (j < k) merged[m++] = other[j++];
    if (keep_low) for (int t = 0; t < k; t++) mine[t] = merged[t];
    else for (int t = 0; t < k; t++) mine[t] = merged[k + t];
    free(other); free(merged);
}

```



```

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, p;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &p);

    int k = (argc > 1) ? atoi(argv[1]) : 8;
    if (p < 2 || (p & (p - 1))) {
        if (rank == 0) fprintf(stderr, "p must be a power of 2.\n");
        MPI_Finalize(); return 1;
    }
    int n = p * k;

    int dims[1] = {p}, periods[1] = {1};
    MPI_Comm ring;
    MPI_Cart_create(MPI_COMM_WORLD, 1, dims, periods, 0, &ring);

    int *mine = malloc((size_t)k * sizeof(int));
    srand(321 + rank * 17);
    for (int i = 0; i < k; i++) mine[i] = rand() % 100000;
    qsort(mine, k, sizeof(int), cmp_int);

    MPI_Barrier(ring);
    double t0 = MPI_Wtime();

    int d = 0; while ((1 << d) < p) d++;
    for (int i = 0; i < d; i++) {
        for (int j = i; j >= 0; j--) {
            int partner = rank ^ (1 << j);
            int ascending = ((rank >> (i + 1)) & 1) == 0;
            int low_side = ((rank >> j) & 1) == 0;
            int keep_low = ascending ? low_side : !low_side;
            compare_split(mine, k, partner, keep_low);
        }
    }

    MPI_Barrier(ring);
    double t1 = MPI_Wtime();

    int *all = NULL;
    if (rank == 0) all = malloc((size_t)n * sizeof(int));
    MPI_Gather(mine, k, MPI_INT, all, k, MPI_INT, 0, ring);
    if (rank == 0) {
        int sorted = 1;
        for (int t = 1; t < n; t++) if (all[t] < all[t - 1]) sorted = 0;
        printf("BLOCK-RING p=%d k=%d n=%d time=%.6f s %s\n",
            p, k, n, t1 - t0, sorted ? "SORTED" : "NOT SORTED");
        free(all);
    }
    free(mine);
    MPI_Comm_free(&ring);
    MPI_Finalize();
    return 0;
}

```

plot_isoefficiency.py

```
#!/usr/bin/env python3
```

```
"""Problem 9.9 - weak-scaling / isoefficiency helper.
```

Runs block_ring under weak scaling (n and p grow together) and plots efficiency vs p. Efficiency $E = T_1 / (p * T_p)$, where T_1 is the 1-process time for the same total n. We approximate T_1 with a sequential sort timing.

```
Usage: python3 plot_isoefficiency.py
```

```

Requires: mpirun, the compiled ./block_ring, matplotlib, numpy.
"""
import subprocess, re, time, random
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

def run_block_ring(p, k):
    out = subprocess.run(
        ["mpirun", "--oversubscribe", "-np", str(p), "./block_ring", str(k)],
        capture_output=True, text=True)
    m = re.search(r"time=([0-9.]+)", out.stdout)
    return float(m.group(1)) if m else None

def seq_sort_time(n):
    a = [random.randint(0, 100000) for _ in range(n)]
    t0 = time.perf_counter(); a.sort(); t1 = time.perf_counter()
    return t1 - t0

# Weak scaling: keep k (work per process) fixed, grow p.
k = 2048
ps = [2, 4, 8, 16]
eff, times = [], []
for p in ps:
    n = p * k
    tp = run_block_ring(p, k)
    t1 = seq_sort_time(n)          # sequential baseline for same n
    e = t1 / (p * tp) if tp and tp > 0 else 0.0
    times.append(tp); eff.append(e)
    print(f"p={p:2d} n={n:6d} Tp={tp:.6f}s T1={t1:.6f}s E={e:.3f}")

plt.figure(figsize=(6,4))
plt.plot(ps, eff, "o-", label="measured efficiency")
plt.xlabel("number of processes p")
plt.ylabel("efficiency E")
plt.title("Ring block bitonic: weak-scaling efficiency")
plt.ylim(0, max(1.0, max(eff)*1.1))
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.savefig("efficiency_vs_p.png", dpi=120)
print("saved efficiency_vs_p.png")

```

Makefile

```

# Problem 9.9 - bitonic sort on a ring
CC      = mpicc
CFLAGS  = -O2 -Wall
LDLIBS  = -lm
all: ring_bitonic block_ring
ring_bitonic: ring_bitonic.c
    $(CC) $(CFLAGS) -o ring_bitonic ring_bitonic.c $(LDLIBS)
block_ring: block_ring.c
    $(CC) $(CFLAGS) -o block_ring block_ring.c
run: all
    mpirun --oversubscribe -np 16 ./ring_bitonic
    for k in 4 8 16 ; do mpirun --oversubscribe -np 4 ./block_ring $$k ; done
clean:
    rm -f ring_bitonic block_ring
.PHONY: all run clean

```

Problem 9.10 — Block-Based Odd-Even Transposition Sort (MPI)

Overview

p processes in a linear array, each holding n/p elements, run p phases alternating even/odd neighbour compare-splits (lower rank keeps the low half). Tested for p in {2,4,8} and n in {16,32,64}; compared against the block bitonic sort of Problem 9.8.

Compilation & Execution

Compile:

```
make
```

Run:

```
mpirun --oversubscribe -np 4 ./oddeven 32
```

```
make run # all p in {2,4,8}, n in {16,32,64}
```

Results

n	p	k	phases	messages	sorted
16	2	8	2	2	yes
32	2	16	2	2	yes
64	2	32	2	2	yes
16	4	4	4	12	yes
32	4	8	4	12	yes
64	4	16	4	12	yes
16	8	2	8	56	yes
32	8	4	8	56	yes
64	8	8	8	56	yes

Table 9.10a: Block-based odd-even transposition sort results.

Analysis

Comparison with block bitonic (9.8): at p=2 and p=4 both need the same messages (2 and 12). At p=8 odd-even needs 56 messages versus bitonic's 48, and the gap widens with p because odd-even uses $O(p)$ phases while bitonic uses $O(\log^2 p)$.

Which is faster depends on the network: bitonic wins by raw operation count, but odd-even uses only nearest-neighbour communication, which can be cheaper on a physical linear/mesh network where bitonic's long-range partners cost extra hops.

Source Code

oddeven.c

```
/* Problem 9.10 - Block-based odd-even transposition sort (MPI, linear array).
 *
 * p processes in a linear array, each holds n/p elements (locally sorted).
 * Run p phases. In even phases, processes (0,1),(2,3),... compare-split; in
 * odd phases, (1,2),(3,4),... compare-split. The lower-rank partner keeps the
 * low half, the higher-rank keeps the high half. After p phases the global
 * sequence is sorted.
 *
 * Test: p in {2,4,8}, n in {16,32,64}. Reports execution time and total
```

```

* communication volume for comparison with block bitonic (Problem 9.8).
*
* Build: mpicc -O2 -o oddeven oddeven.c
* Run:   mpirun -np 4 ./oddeven 32
*/
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

static int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}

static void compare_split(int *mine, int k, int partner, int keep_low,
                          long *msgs, long *bytes) {
    int *other = malloc((size_t)k * sizeof(int));
    int *merged = malloc((size_t)2 * k * sizeof(int));
    MPI_Sendrecv(mine, k, MPI_INT, partner, 0, other, k, MPI_INT, partner, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    *msgs += 1; *bytes += (long)k * sizeof(int);
    int i = 0, j = 0, m = 0;
    while (i < k && j < k) merged[m++] = (mine[i] <= other[j]) ? mine[i++] : other[j++];
    while (i < k) merged[m++] = mine[i++];
    while (j < k) merged[m++] = other[j++];
    if (keep_low) for (int t = 0; t < k; t++) mine[t] = merged[t];
    else          for (int t = 0; t < k; t++) mine[t] = merged[k + t];
    free(other); free(merged);
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, p;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &p);

    int n = (argc > 1) ? atoi(argv[1]) : 32;
    if (n % p != 0) {
        if (rank == 0) fprintf(stderr, "n (%d) must be a multiple of p (%d).\n", n, p);
        MPI_Finalize(); return 1;
    }
    int k = n / p;

    int *mine = malloc((size_t)k * sizeof(int));
    srand(909 + rank * 23);
    for (int i = 0; i < k; i++) mine[i] = rand() % 10000;
    qsort(mine, k, sizeof(int), cmp_int); /* local sort */

    long msgs = 0, bytes = 0;
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    for (int phase = 0; phase < p; phase++) {
        int partner;
        if (phase % 2 == 0) /* even phase: (0,1)(2,3)... */
            partner = (rank % 2 == 0) ? rank + 1 : rank - 1;
        else /* odd phase: (1,2)(3,4)... */
            partner = (rank % 2 == 0) ? rank - 1 : rank + 1;
        if (partner < 0 || partner >= p) continue; /* boundary idles */
        int keep_low = (rank < partner); /* lower rank keeps low half */
        compare_split(mine, k, partner, keep_low, &msgs, &bytes);
    }

    MPI_Barrier(MPI_COMM_WORLD);
    double t1 = MPI_Wtime();

```

```

int *all = NULL;
if (rank == 0) all = malloc((size_t)n * sizeof(int));
MPI_Gather(mine, k, MPI_INT, all, k, MPI_INT, 0, MPI_COMM_WORLD);

long tot_msgs, tot_bytes;
MPI_Reduce(&msgs, &tot_msgs, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
MPI_Reduce(&bytes, &tot_bytes, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0) {
    int sorted = 1;
    for (int t = 1; t < n; t++) if (all[t] < all[t - 1]) sorted = 0;
    printf("ODD-EVEN n=%d p=%d k=%d time=%.6f s messages=%ld comm_bytes=%ld %s\n",
        n, p, k, t1 - t0, tot_msgs, tot_bytes, sorted ? "SORTED" : "NOT SORTED");
    free(all);
}
free(mine);
MPI_Finalize();
return 0;
}

```

Makefile

```

# Problem 9.10 - block-based odd-even transposition sort
CC      = mpicc
CFLAGS  = -O2 -Wall
all: oddeven
oddeven: oddeven.c
    $(CC) $(CFLAGS) -o oddeven oddeven.c
run: all
    for p in 2 4 8 ; do for n in 16 32 64 ; do \
        mpirun --oversubscribe -np $$p ./oddeven $$n ; done ; done
clean:
    rm -f oddeven
.PHONY: all run clean

```

9.10 vs 9.8 — message-count comparison

p	odd-even msgs	block-bitonic msgs
2	2	2
4	12	12
8	56	48

Table 9.10b: Block odd-even vs block bitonic — bitonic scales better as p grows.